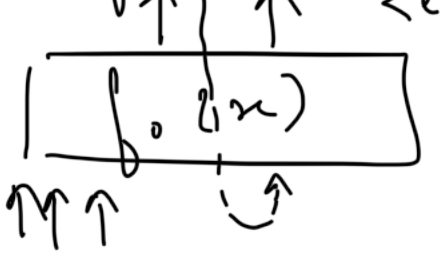
LLM

Week-4

L1 Decoding Strategies

- Transformer as a func^{n}



predicting the next Token → tells when to stop.

- iterate & generate full story (if you want).
- we are currently using simple fine-tuning on basic NLP task.
- discourage degenerative Task encourage to be creative accelerate the gener^{n} of tokens
- we will learn about decoding strategies that will help us in being more creative.

L2 Exhaustive Search

- deterministic decoding strategy ⟹ exhaustive (brute-force method)

- we are given vocab of 5 words.

$$\underbrace{|V| \quad |V| \quad |V| \quad |V| \quad |V|}_{|V|^5}$$

- in this, we exhaustively search for all possible sequences with the associated probabilities & outputs the seq. with highest probability.
  → each seq. $(p(\cdot) = 0.1)$ has a probab. over that vocab.

$$p(x_1, x_2 \ldots x_n) = p(x_1) \cdot p(x_2 | x_1)$$

...

Eg. $p(I, \text{like}, \text{cold}, \text{coffee}) = p(I) \cdot p(\text{like} | I)$.

$$= p(\text{cold} | I, \text{like}) \cdot$$

$$= p(\text{coffee} | I, \text{like}, \text{cold})$$

  → we always use the chain rule to acalcute the probabilities to each sentence.

- If $v = |40000|$, then it becomes an infeasible task to run for so many iterations.

L3 Greedy & Beam Search

- on other extreme, we can do a greedy search. At each time step, we always output the token with the highest probability (greedy).
- as opposed to exhaustive search, this greedy approach is better.
- same problem – deterministic, always get the same seq. of words.
- some limits ⇒
  1. is this the most likely seq.?
  2. what if we want to get a variety of seq. of same len.
  3. what if we take second highest probability word in the 1st time step?
     ⇒ The conditional probab. in subsequent steps will Δ.
  4. another pt. to note is → greedily selecting a token with max. probab. at each time step does not always give the seq. with max. total probab.
     ⇒ Is it possible to do this at every time step?

- Beam Search - instead of considering probabs. at every time step, we consider only top-k tokens. (k can be chosen).
- this is somewhere btw. the greedy & the exhaustive search.
- Beam req. $k \times |V|$ computa^{n} at each time step.
- The param k is c/d the beam size. It is approxima^{n} to exhaustive search. If $k=1$, it becomes greedy search.
- We have k sequences at the end of time step T & outputs the seq. which has highest probabs.
- divide the seq. probabs. by the len. (otherwise long sentences will have less probabs.). Then output the seq. with highest probabs.
- both are degenerating
- latency for greedy < beam search.
- neither of them have creative outputs. ⇒ surprise = uncertainity
- beam is highly suitable for tasks like transla^{n} & summariza^{n}.

L4 Sampling Strategies - Top k

- output is not deterministic
- at every time-step, consider top-k tokens from the probab. distribⁿ Probab. of top-k would be normalized before next token.
- sample a token from top-k tokens. Then generate the sequence using top-k sampling.
- In this way, we get diff. tokens every time ⇒ thus, creativity comes.

L5 Top-p

- what should be the optimal value of k?
  ↳ there is a problem in a peaky distribⁿ. In some cases, if we fix up the value of k, then we are actually missing out on other equally probable tokens in flat distribⁿ. For peaky ones, it leads to meaningless sentences.

- Sol^n 1 → Low Temp. Sampling

given $u_{1:|V|}$ & T (temp. param.)

compute the probabilities as -

$$P(x = u_l \mid x_{1:i-1}) = \frac{\exp\left(\frac{u_l}{T}\right)}{\sum_{l'} \exp\left(\frac{u_{l'}}{T}\right)}$$

low T = skewed distribu^n = low creativity

- Sol^n 2 → Nucleus Sampling (Top - p) (better)

probability mass

gives me all those tokens that accounts for somewhere around 60% of the probability mass.

It will automatically accomadate both flatter & skewed distributions.

Process —

1. sort probab. in desc.
2. set value for the param p $0 < p < 1$
3. sum the probab. from top tokens
4. stop at threshold.

it will be similar to top-k (now k being dynamic)

L6 BERT - Masked Language Modelling

- previously we were looking at decoder based models

  GPT - causal language model
  ↳ the represent^n is learned in an unidirectional (Left to right) way. It is a natural fit for tasks like text genera^n. But what abt. NER tasks?

- we need to look at both direc^{ns} (surrounding words) to predict the masked words (like CBOW). This is c/d masked language modelling. Obv. decoder can't help as it is inherently unidirectional.

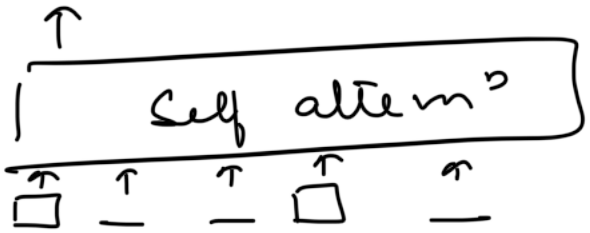
- Let's use encoder -

$$P(y_i \mid x_1, x_i = [\text{mask}], \ldots, x_T) = ?$$

↙ we need to find this, thus, max. the probab. of the masked word.

It is assumed that the predicted tokens are independent of each other. We need to find the ways to mask the words in the input text.

- MLM



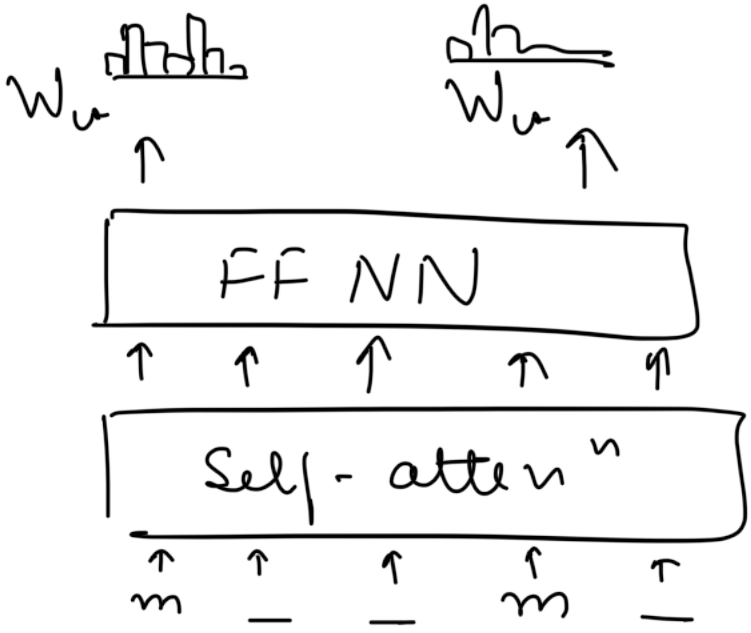
we know that each word attends to every other word in the sequence in Self attenⁿ. We have to randomly mask and predict few words. Also, can we mask the attenⁿ wts of the words to be masked as in CLM?

We can't do that, we want model to learn that there are blanks. We think the mask as a noise that is corrupting the original input text. Then the model has a Task to recover the original token. This is what is also done in denoising of auto

encoders. Thus, MLM is old pre-training denoising objective.

- A simple way to bring <mask> token in $|V|$.

$$L_1 = -\log(\hat{y}_1) \qquad L_2 = -\log(\hat{y}_2)$$



$$L = \frac{1}{N} \sum_{x \in M} -\log(\hat{y}_i)$$

- only a set of M mask to predicted (gen. 15% of input seq), hence, cross entropy loss for those predic^n. If we < 15% then long time to convergence & inefficient training.
- BERT base had 12 layers with 12 attn heads per layer. 15% words are masked. Out of it, 80% have <mask>

token, 10% random words, 10% retained.

- The spl. mask token is not a part of dataset while preparing for down-stream tasks.

L7 Next Sentence Predic"

- NSP – extend the input with the pair of sentences (A, B) & <sep> token to show where they are separated. <CLS> to start with.

  input : Sentance : (A, B)
  label : isNext

- In 50% of instance, B is naturally followed A. In other 50%, B is not Next. Pretraining with NSP improves performance of QA & other similar tasks significantly.
- Along with token embeddings, they had segment embedding (2×dim). And position embedding (2 × dim).

Obj. min. $L = \frac{1}{n} \sum_{x \in M} -\log(\hat{y}_i) + L_{cls}$

L8 Pre-Training & Parameters

- pre-training
  dataset – 800M (bookcorpus) +
  2500M (wikipedia)
  vocab size – 30,000
  context len – ≤ 512 tokens
  no. of layers – 12 (base) / 24 (large)
  hidden size (d_{model}) – 768 / 1024
  no. of attn heads – 12 / 16
  intermediate size (4H) – 3072 / 4096
- param calc.
  embedding

  Token — $30000 \times 768 = 23M$
  segment — $2 \times 768 \approx 15k$
  posn — $512 \times 768 = 0.4M$

  Total = 23.4 M

  for single layer,
  self attn (no bias) — W_K, W_Q, W_V, A
  $(768 \times 64 \times 3) \times 12 = 1.7M$

  $768 \times 768 = 0.6M$

FFN

$(768 \times 3072) \times 2 + (3072 + 768) = 4.7\,M$

Total = 7 M

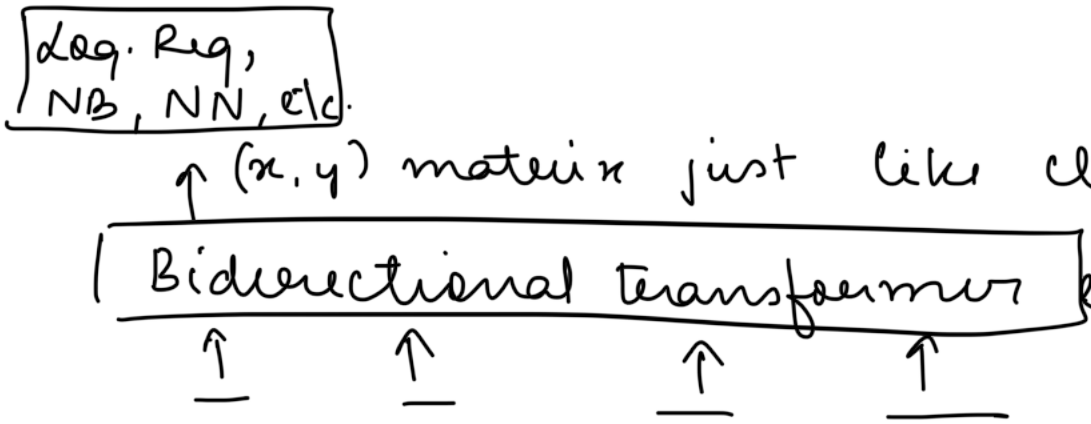
12 x 7 (layers) = 84 M

Total params = 84 + 23.4 = 107.4 M

L9 Applying to Downstream Tasks

- we have now completed the pre-training task. we now have to prepare our model for down stream tasks. earlier there were 2 approaches used —
  1. BERT as feature extractor
  2. fine-tuning BERT
- Classificⁿ - feature-based
  - ⇒ Take a sentence of len ≤ 512 & feed it as an input to BERT (with app. padding as & when req.)
  - → Take the output representⁿ (output from the last encoder) as a feature vector for the next seq.

→ such a representa" received would be superior to just concatenating representa~ of indi. words



→ during the process, all the params of the BERT model is frozen & only the classifica" head is trained from scratch.

- Classifica" – Fine-tuning
  → take the same sentence as before.
  → in this, we update the BERT wts as well. (the major & only diff. btw. above method)
  → general obs. → model used in the classifica" head converges quickly with a less no. of labelled training samples than the feature-based approach.
- extractive question-answer
  ↳ using both labelled & unlabelled data.

$S \rightarrow$ start vector of size of h_i
prob. of i^{th} word being the 1^{st} token

$$S_i = \frac{\exp(S \cdot h_i)}{\sum_{j=1}^{25} \exp(S \cdot h_j)}$$

similarly, we have E denotes the end vector of size of h_i.
constraints like $E > S$ will be obviously true.